LOGO! Soft Comfort — Beginner's Programming Guide

> 📄 **Print-ready PDF:** `docs/pdf/logo-soft-comfort-guide.pdf`

This guide walks you through programming the Siemens LOGO! 12/24RCE PLC for the Juniper Test Station, step by step, with no prior PLC or ladder logic experience required. By the end you will have a working FBD program uploaded to the LOGO! that enforces hardware safety on every output.

What Is FBD (Function Block Diagram)?

FBD is one of four standard PLC programming languages. Instead of writing code, you place graphical blocks on a canvas and draw wires between them — like a circuit diagram for logic.

Each block represents a logical operation (AND, OR, NOT, a flip-flop, a timer, etc.). Signals flow left to right: inputs arrive on the left, the block does its calculation, and the result exits on the right.

```
Signal A ──┐
           ├──[ AND ]──► Output
Signal B ──┘
```

In LOGO! Soft Comfort, the canvas is called a **network**. Our program has six networks — one per logical function.

Software Installation

1. Insert the CD that came with the LOGO!.
2. Run `LOGO_Soft_Comfort_V8.x_Setup.exe`.
3. Accept defaults. The software installs to `C:\Program Files\Siemens\LOGO!SoftComfort8`.
4. Launch **LOGO! Soft Comfort** from the Start menu.
5. On first run, accept the licence and dismiss the welcome screen.

> **Alternative — free download:** If you have lost the CD, LOGO! Soft Comfort 8 is available from the Siemens support portal at `support.industry.siemens.com` (search "6ED1058-0BA08-0YA1"). You will need a free Siemens ID account.

Creating a New Project

1. Click **File** → **New** (or press `Ctrl+N`).
2. In the **Select device** dialog, expand the **LOGO! 8** family.
3. Select **LOGO! 12/24RCE** (part number 6ED1052-1MD08-0BA1).
4. Click **OK**.
5. The editor opens with a blank canvas. The left panel shows the **block catalog**.

Save the project now: `File → Save As → Juniper_Test_Station.lsc`

Understanding the Editor

```
┌──────────────────────────────────────────────────────────────┐
│  Menu bar:  File | Edit | View | Format | Network | Tools    │
├────────────┬─────────────────────────────────────────────────┤
│  Block     │                                                  │
│  catalog   │             FBD canvas (networks)                │
│  (left     │                                                  │
│  panel)    │  ← this is where you place and wire blocks       │
│            │                                                  │
├────────────┴─────────────────────────────────────────────────┤
│  Status bar / output messages                                 │
└──────────────────────────────────────────────────────────────┘
```

Key toolbar buttons:

Icon / shortcut	Action
Green play ▶	Simulate program on-screen (no hardware needed)
Upload ↑	Transfer program to connected LOGO!
Download ↓	Read program back from LOGO!
Magnifying glass	Online monitor (watch live values when connected)

I/O Reference — Our Rig Wiring

Before programming, familiarise yourself with what each terminal does. These match the physical wiring described in `docs/wiring-guide.md`.

Physical inputs (left side of LOGO! terminal block):

Terminal	Signal	Switch type	LOGO! reads HIGH when…
I1	E-stop button	NC (normally closed)	Button is NOT pressed (safe)
I2	Chamber door reed switch	NC	Door is CLOSED
I3	120 °C bimetallic thermostat	NO (normally open)	Temperature has TRIPPED (fault!)
I4	Green start button	NO	Button IS pressed
I5	Red stop button	NC	Button is NOT pressed (safe)

Physical outputs (right side of terminal block):

Terminal	Signal	What it controls
Q1	HEATER_RELAY	SSR that switches the 120 V AC heater cartridge
Q2	SERVO_POWER	24 V→5 V BEC rail enabling the vent servo motors
Q3	DUT_RELAY	Power relay for the Device Under Test
Q4	ALARM_BEACON	External 24 V audible/visual alarm

Internal flags (software-only, no terminal):

Flag	Name	Purpose
M1	SafetyGate	HIGH only when ALL safety conditions are met
M2	RunLatch	HIGH after start button pressed; held by SR flip-flop
M3	SW_HEATER	Written by RPi via Modbus — "software heater request"
M4	SW_SERVO	Written by RPi via Modbus — "software servo request"
M5	SW_DUT	Written by RPi via Modbus — "software DUT request"

> **Why M3/M4/M5?** The Raspberry Pi cannot directly energise Q1/Q3 — it writes M3/M4/M5 via Modbus as "requests". The LOGO! program then decides whether to actually energise the relay, based on whether the safety gate is clear. This means a crashed RPi or software bug cannot bypass the E-stop.

Network 1 — Safety Gate (M1)

What it does: Combines all three hardware safety conditions into a single flag, M1. If ANY safety condition is violated, M1 goes LOW and every protected output shuts off within one scan cycle (~100 ms).

Logic: `M1 = I1 AND I2 AND (NOT I3)`

How to enter this in LOGO! Soft Comfort

Step 1 — Add the first input block (I1): 1. In the block catalog (left panel), expand **Constants/Connectors**. 2. Drag **Input I1** onto the canvas, roughly at position (2, 3). 3. You will see a small block labelled `I1` appear.

Step 2 — Add I2: 1. Drag **Input I2** below I1 at position (2, 5).

Step 3 — Add I3 with NOT (inversion): 1. Drag **Input I3** below I2 at position (2, 7). 2. Right-click the I3 block → **Properties** → tick **Invert input** (or click the small circle at the output pin to toggle inversion). - The inverted pin shows a small circle `o` on its output wire. - This means: when I3 is HIGH (thermostat tripped), the signal entering the AND gate is LOW.

Step 4 — Add an AND gate: 1. In the block catalog, expand **Basic functions (GF)**. 2. Drag a **3-input AND** gate to position (5, 5) on the canvas. - LOGO! Soft Comfort labels this block `&` (standard IEC notation for AND).

Step 5 — Wire the inputs to the AND gate: 1. Hover over the output pin of the I1 block until the cursor changes to a crosshair. 2. Click and drag to the top input pin of the AND gate. A line appears. 3. Repeat for I2 → middle AND input. 4. Repeat for I3 (inverted) → bottom AND input.

Step 6 — Add the M1 output flag: 1. In the block catalog, expand **Constants/Connectors**. 2. Drag **Flag M1** to position (8, 5). 3. Wire the AND gate output → M1 input.

Completed Network 1 should look like this:

```
I1 ────────────────────────────┐
                               ├──[&]──► M1
I2 ────────────────────────────┤
                               │
I3 ──o (inverted) ─────────────┘

(o = inverted signal — when I3 HIGH → LOW enters the AND gate)
```

Step 7 — Label the network: 1. Double-click the network title bar and type: `Safety Gate (M1)`.

Network 2 — Run Latch (M2)

What it does: Creates a latching "run" state. Pressing the green start button (I4) sets M2 HIGH; it stays HIGH even after the button is released. Pressing the red stop button (I5 NC, so I5 goes LOW when pressed) or losing the safety gate resets M2.

Logic: SR flip-flop with `S = I4`, `R = (NOT I5) OR (NOT M1)`

How to enter this in LOGO! Soft Comfort

Step 1 — Add an SR flip-flop: 1. In the block catalog, expand **Special functions (SF)**. 2. Drag **Latching relay (SR)** onto a new network canvas area (use `Network → Add network` from the menu to create Network 2, or just use the same canvas below Network 1 if there is room). 3. The SR block has two inputs: **S** (set) at the top, **R** (reset) at the bottom, and output **Q**.

Step 2 — Wire the Set input: 1. Add `Input I4` to the left of the SR block. 2. Wire I4 → S input of the SR flip-flop.

Step 3 — Build the Reset condition: The reset is `NOT(I5) OR NOT(M1)`. Two conditions must be OR'd together.

1. Add `Input I5` to the canvas. Right-click → **Properties** → tick **Invert input** (so it becomes "NOT I5").
2. Add `Flag M1` to the canvas. Right-click → **Properties** → tick **Invert input** (so it becomes "NOT M1").
3. Add a **2-input OR gate** from GF → Basic functions.
4. Wire NOT(I5) → top OR input.
5. Wire NOT(M1) → bottom OR input.
6. Wire OR output → **R** input of the SR flip-flop.

Step 4 — Add output: 1. Add `Flag M2` to the right. 2. Wire SR Q output → M2.

Completed Network 2:

```
I4 ──────────────────────────────────────► S ┐
                                              │SR FF├──► M2
I5 ──o ──┐                                    │
         ├──[≥1]──────────────────────────► R ┘
M1 ──o ──┘

(≥1 = OR gate; o = inverted)
```

Label: `Run Latch (M2)`

Network 3 — Q1 Heater Relay

What it does: Energises the heater SSR relay. Requires ALL three conditions: safety gate clear, run latch active, and RPi software requesting heater ON.

Logic: `Q1 = M1 AND M2 AND M3`

How to enter this in LOGO! Soft Comfort

Step 1: Add a new network. Label it `Q1 — Heater Relay`.

Step 2: Place three input blocks: `Flag M1`, `Flag M2`, `Flag M3`.

Step 3: Add a **3-input AND gate** from GF → Basic functions.

Step 4: Wire: - M1 → top AND input - M2 → middle AND input - M3 → bottom AND input

Step 5: Add `Output Q1` and wire the AND gate output to it.

```
M1 ──┐
     ├──[&]──► Q1
M2 ──┤
     │
M3 ──┘
```

Network 4 — Q2 Servo Power Relay

What it does: Powers the 24 V BEC rail for the servo motors. Requires safety gate, but NOT the run latch — servos may be moved to their home position during idle or cool-down.

Logic: `Q2 = M1 AND M4`

How to enter this in LOGO! Soft Comfort

Step 1: Add a new network. Label it `Q2 — Servo Power`.

Step 2: Place `Flag M1` and `Flag M4`.

Step 3: Add a **2-input AND gate**.

Step 4: Wire M1 and M4 into the AND inputs. Wire AND output → `Output Q2`.

```
M1 ──┐
     ├──[&]──► Q2
M4 ──┘
```

Network 5 — Q3 DUT Power Relay

What it does: Energises the Device Under Test power relay. Same three-condition requirement as the heater.

Logic: `Q3 = M1 AND M2 AND M5`

How to enter this in LOGO! Soft Comfort

Identical procedure to Network 3, substituting `M5` for `M3` and `Q3` for `Q1`.

```
M1 ─┐
    ├─[&]──► Q3
M2 ─┤
    │
M5 ─┘
```

Label: `Q3 — DUT Power`

Network 6 — Q4 Alarm Beacon

What it does: Fires the alarm beacon immediately when any safety fault occurs, regardless of software state. Cannot be silenced from software — only clearing the hardware fault turns it off.

Logic: `Q4 = (NOT I1) OR (NOT I2) OR I3`

How to enter this in LOGO! Soft Comfort

Step 1: Add a new network. Label it `Q4 — Alarm Beacon`.

Step 2: Add three input blocks: `Input I1` (inverted), `Input I2` (inverted), `Input I3` (NOT inverted).

Step 3: Add a **3-input OR gate** from GF → Basic functions.

Step 4: Wire: - NOT(I1) → top OR input - NOT(I2) → middle OR input - I3 → bottom OR input

Step 5: Wire OR output → `Output Q4`.

```
I1 ──o ──┐
         ├─[≥1]──► Q4
I2 ──o ──┤
         │
I3 ────── ┘

(fires when E-stop tripped, door opened, OR overtemp active)
```

Configuring Ethernet & Modbus TCP

The RPi communicates with the LOGO! over Modbus TCP. You must configure this before the software can connect.

Step 1 — Assign a static IP to the LOGO!: 1. In LOGO! Soft Comfort, go to **Tools** → **Ethernet connections**. 2. Click **Add** to create a new connection entry. 3. Set: - **IP address:** `192.168.1.100` - **Subnet mask:** `255.255.255.0` - **Gateway:** `192.168.1.1` (or your router's IP) 4. Click **OK**.

Step 2 — Enable Modbus TCP server: 1. Go to **Tools** → **Ethernet** → **Modbus server**. 2. Tick **Enable Modbus server**. 3. **Port:** `502` (default, leave as-is). 4. Click **OK**.

Step 3 — Verify network settings: 1. Go to **Tools** → **Network settings**. 2. Ensure the LOGO!'s IP is `192.168.1.100` and Modbus is ticked.

Transferring the Program to the LOGO!

Before transferring: - The LOGO! must be powered on. - Your PC (running LOGO! Soft Comfort) must be on the same subnet (e.g. `192.168.1.x`). - Either connect with the USB programming cable (the blue cable included in the box) OR use Ethernet.

Via USB programming cable (simplest for first upload):

1. Connect the blue USB programming cable between the LOGO!'s front-panel USB port and your PC.
2. In LOGO! Soft Comfort, click **Tools** → **Transfer** → **PC** → **LOGO!** (the upload arrow ↑).
3. In the connection dialog, select **USB** as the interface.
4. Click **OK**. The transfer takes about 10 seconds.
5. You will see: `Transfer complete` in the status bar.
6. The LOGO! display will show **STOP** — press the front-panel `ESC` button and then navigate to **Start** to put it into **RUN mode**.

Via Ethernet (after first IP assignment):

1. The first time, you must set the IP via USB (see step above), then switch to Ethernet.
2. In LOGO! Soft Comfort, go to **Tools** → **Transfer** → **PC** → **LOGO!** → select **Ethernet**.
3. Enter IP address `192.168.1.100`, click **Search** to verify it responds.
4. Click **Transfer**.

Setting LOGO! to RUN Mode

After transfer, the LOGO! is in STOP mode. To start the FBD program:

Option A — Front panel: 1. On the LOGO! unit, press the front-panel button (▶/◀ or `ESC` depending on the model variant). 2. Navigate the small display to `Start` and press OK. 3. The display will show `RUN` or your program's first output state.

Option B — From LOGO! Soft Comfort: 1. Click **Tools** → **Transfer** → **Start LOGO!** (or the ▶ button in the transfer toolbar).

Online Monitoring (Testing Without Hardware)

Before wiring anything up, you can simulate inputs and watch outputs change in Soft Comfort:

1. With the LOGO! connected and in RUN, click the **Online monitor** button (magnifying glass icon).
2. The canvas goes live — current signal states are shown as colour highlights:
3. **Green** = signal is HIGH / TRUE
4. **Grey** = signal is LOW / FALSE
5. In the online monitor, you can **force** input values by right-clicking an input block → **Set value** → toggle it ON/OFF.
6. Test the safety gate: force I1 to LOW (E-stop pressed) → watch M1 go grey → watch Q1, Q3 go grey.
7. Test the alarm: force I1 to LOW → Q4 should go green immediately.

Verification Checklist

Run this after the first successful transfer, before wiring any high-voltage loads.

Test	Expected result	Pass?
LOGO! powered, in RUN mode	Display shows `RUN` or output states	☐
Modbus TCP responds	`python -c "from plc.logo_driver import LogoDriver; plc=LogoDriver(); plc.connect('192.168.1.100'); print(plc.ping())"` prints `True`	☐
I1 reads HIGH (E-stop released)	`plc.get_named_input("ESTOP")` returns `True`	☐
I2 reads HIGH (door closed)	`plc.get_named_input("DOOR_INTERLOCK")` returns `True`	☐
I3 reads LOW (cool — under 120 °C)	`plc.get_named_input("OVERTEMP_CUT")` returns `False`	☐
M1 (safety gate) is HIGH	Online monitor shows M1 green	☐
Press I4 (start button)	M2 latches HIGH (stays green after button released)	☐
RPi writes M3 (SW_HEATER) via Modbus	`plc.set_named_output("HEATER_RELAY", True)` → Q1 relay clicks	☐
Press E-stop (I1 goes LOW)	M1 goes LOW → Q1 and Q3 de-energise immediately → Q4 alarm fires	☐
Release E-stop, press stop (I5 opens)	M2 resets, Q1/Q3 stay off until I4 is pressed again	☐
RPi writes M3 with E-stop pressed	Q1 does NOT energise (safety gate blocks it)	☐

Troubleshooting

"Modbus TCP connect failed" - Verify LOGO! is in RUN mode (not STOP). - Check IP address in `rig_config.json` matches what you programmed (default: `192.168.1.100`). - Confirm your PC/RPi is on the same subnet (`192.168.1.x / 255.255.255.0`). - Ping the LOGO!: `ping 192.168.1.100`. If no reply, recheck the Ethernet cable or re-transfer the IP settings via USB.

"Q1 relay does not click when I write M3" - Check M1 (safety gate) is HIGH in the online monitor. If LOW, a safety input is faulted. - Check M2 (run latch) is HIGH. If not, press the physical start button I4. - Confirm the I/O wiring matches the terminal assignments in this guide.

"LOGO! display shows STOP after power-up" - LOGO! 8 defaults to STOP after power cycle unless you enable auto-start. - In LOGO! Soft Comfort → **Tools** → **Parameters** → **Start behaviour** → set to **Run**. - Re-transfer the program.

"Soft Comfort cannot find the LOGO! over Ethernet" - The LOGO!'s IP is only programmable via the USB cable on first setup. - Connect the blue USB cable, open **Tools** → **Transfer** with USB selected, re-transfer the program with the Ethernet settings, then switch to Ethernet for future use.